

LAPORAN JOBSHEET 11
LINKED LIST



Disusun oleh:
NAURA FADHILLA ADITYA PUTRI
254107020007
TI-1C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

A. 2.1 Pembuatan Single Linked List

1. Hasil program di Mahasiswa20.java

```
mingguli1 > Mahasiswa20.java > Language support for Java(TM) by Red Hat > Mahasiswa20 > tampilkanInformasi()
1 package Minggu11;
2
3 public class Mahasiswa20 {
4     String nim;
5     String nama;
6     String kelas;
7     double ipk;
8
9     public Mahasiswa20() {
10    }
11
12    public Mahasiswa20(String nim, String nama, String kelas, double ipk) {
13        this.nim = nim;
14        this.nama = nama;
15        this.kelas = kelas;
16        this.ipk = ipk;
17    }
18
19    void tampilkanInformasi() {
20        System.out.println("NIM : " + nim);
21        System.out.println("Nama : " + nama);
22        System.out.println("Kelas : " + kelas);
23        System.out.println("IPK : " + ipk);
24        System.out.println(x: "-----");
25    }
26 }
27
```

2. Hasil program di Node20.java

```
mingguli1 > Node20.java > java > Node20 > Node20(Mahasiswa20 data, Node20 next)
1 package Minggu11;
2
3 public class Node20 {
4     Mahasiswa20 data;
5     Node20 next;
6
7     public Node20(Mahasiswa20 data, Node20 next) {
8         this.data = data;
9         this.next = next;
10    }
11
12 }
13
```

3. Hasil program di SingleLinkedList20.java

```
Minggu11 > SingleLinkedList20.java > Language Support for Java(TM) by Red Hat > SingleLinkedList20 > addFirst(Mahasiswa20)
1  package Minggu11;
2
3  public class SingleLinkedList20 {
4      Node20 head;
5      Node20 tail;
6
7      // method empty
8      boolean isEmpty() {
9          return (head == null);
10     }
11
12     // method print
13     void print() {
14         if (!isEmpty()) {
15             Node20 tmp = head;
16             System.out.println(x: "Isi Linked List:");
17             while (tmp != null) {
18                 tmp.data.tampilkanInformasi(); // Memanggil method tampil milik Mahasiswa20
19                 tmp = tmp.next;
20             }
21         } else {
22             System.out.println(x: "Linked list kosong");
23         }
24     }
25
26     // method addFirst (Memasukkan di paling depan/head)
27     void addFirst(Mahasiswa20 input) {
28         Node20 ndInput = new Node20(input, next: null);
29         if (isEmpty()) {
30             head = ndInput;
31             tail = ndInput;
32         } else {
33             ndInput.next = head; // Hubungkan node baru ke head lama
34             head = ndInput;    // Pindahkan head ke node baru
35         }
36     }
37
38     // method addLast (Memasukkan di paling belakang/tail)
39     void addLast(Mahasiswa20 input) {
40         Node20 ndInput = new Node20(input, next: null);
41         if (isEmpty()) {
42             head = ndInput;
43             tail = ndInput;
44         } else {
45             tail.next = ndInput; // Hubungkan tail lama ke node baru
46             tail = ndInput;    // Pindahkan tail ke node baru
47         }
48     }
49 }
```

```

3 public class SingleLinkedList20 {
50 // method insertAfter
51 void insertAfter(String key, Mahasiswa20 input) {
52     Node20 ndInput = new Node20(input, next: null);
53     Node20 temp = head;
54     while (temp != null) {
55         if (temp.data.nama.equalsIgnoreCase(key)) {
56             ndInput.next = temp.next;
57             temp.next = ndInput;
58             if (ndInput.next == null) { // Jika disisipkan di paling akhir
59                 tail = ndInput;
60             }
61             break;
62         }
63         temp = temp.next;
64     }
65 }
66
67 // method insertAt
68 void insertAt(int index, Mahasiswa20 input) {
69     if (index < 0) {
70         System.out.println(x: "Indeks tidak valid");
71     } else if (index == 0) {
72         addFirst(input);
73     } else {
74         Node20 temp = head;
75         for (int i = 0; i < index - 1; i++) {
76             if (temp != null) {
77                 temp = temp.next;
78             }
79         }
80         if (temp != null) {
81             temp.next = new Node20(input, temp.next);
82             if (temp.next.next == null) {
83                 tail = temp.next;
84             }
85         }
86     }
87 }
88

```

4. Hasil program di SLLMain20.java

```

public class SLLMain20 {
    public static void main(String[] args) {

        // Inisialisasi objek mahasiswa
        Mahasiswa20 mhs1 = new Mahasiswa20(nim: "21212203", nama: "Dirga", kelas: "4D", ipk: 3.6);
        Mahasiswa20 mhs2 = new Mahasiswa20(nim: "22212202", nama: "Cintia", kelas: "3C", ipk: 3.5);
        Mahasiswa20 mhs3 = new Mahasiswa20(nim: "23212201", nama: "Bimon", kelas: "2B", ipk: 3.8);
        Mahasiswa20 mhs4 = new Mahasiswa20(nim: "24212200", nama: "Alvaro", kelas: "1A", ipk: 4.0);

        // 1. Cetak awal (Linked list kosong)
        sll.print();

        // 2. Alvaro dimasukkan pertama kali menggunakan addFirst
        sll.addFirst(mhs4);
        sll.print();

        // 3. Dirga dimasukkan di akhir menggunakan addLast
        sll.addLast(mhs1);
        sll.print();

        // 4. Masukkan Bimon SETELAH Dirga
        sll.insertAfter(key: "Dirga", mhs3);

        // 5. Masukkan Cintia di INDEKS KE-2 (posisi setelah Dirga sebelum Bimon)
        sll.insertAt(index: 2, mhs2);

        // 6. Cetak hasil akhir struktur list
        sll.print();
    }
}

```

Hasil Output di Main

```
Linked list kosong
Isi Linked List:
NIM : 24212200
Nama : Alvaro
Kelas : 1A
IPK : 4.0
-----
Isi Linked List:
NIM : 24212200
Nama : Alvaro
Kelas : 1A
IPK : 4.0
-----
NIM : 21212203
Nama : Dirga
Kelas : 4D
IPK : 3.6
-----
Isi Linked List:
NIM : 24212200
Nama : Alvaro
Kelas : 1A
IPK : 4.0
-----
NIM : 21212203
Nama : Dirga
Kelas : 4D
IPK : 3.6
-----
NIM : 22212202
Nama : Cintia
Kelas : 3C
IPK : 3.5
-----
NIM : 23212201
Nama : Bimon
Kelas : 2B
IPK : 3.8
-----
```

Pertanyaan:

- a. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?

Jawab: Karena pada saat program pertama kali dijalankan, Linked List belum memiliki data/node sama sekali.

- variabel head masih bernilai null
- belum ada mahasiswa yang ditambahkan ke dalam linked list

- b. Jelaskan kegunaan variable temp secara umum pada setiap method!

Jawab: Variabel temp pada Linked List digunakan sebagai penunjuk sementara (temporary pointer/reference) untuk menelusuri node-node di dalam linked list tanpa mengubah data asli. Kegunaan temp yaitu:

- Menelusuri isi Linked list, Digunakan untuk berpindah dari satu node ke node berikutnya.

- Menampilkan data, Saat method print/display dijalankan, temp digunakan untuk membaca semua node satu per satu tanpa mengubah head.
- Mencari data tertentu, Pada method search, temp dipakai untuk mengecek setiap node sampai data ditemukan.
- Membantu proses tambah/hapus node, Digunakan untuk mencari posisi node terakhir atau node sebelum node yang akan dihapus.
- Menjaga data asli tetap aman, Jika langsung memakai head untuk traversal, posisi awal linked list bisa berubah.

c. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawab: modifikasi program dengan menambahkan Scanner untuk input dinamis

```
package Minggu11;
import java.util.Scanner;
public class SLLMain20 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        SingleLinkedList20 sll = new SingleLinkedList20();
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "=== INPUT DATA MAHASISWA KE LINKED LIST ===");

        System.out.print(s: "NIM: ");
        String nim = sc.nextLine();
        System.out.print(s: "Nama: ");
        String nama = sc.nextLine();
        System.out.print(s: "Kelas: ");
        String kelas = sc.nextLine();
        System.out.print(s: "IPK: ");
        double ipk = sc.nextDouble();

        Mahasiswa20 mhs = new Mahasiswa20(nim, nama, kelas, ipk);
        sll.addLast(mhs);
        sll.print();

        sc.close();
    }
}
```

B. 2.2 Modifikasi Elemen pada Single Linked List

1. Hasil Modifikasi program di SingleLinkedList20.java

```
// method getData
void getData(int index) {
    Node20 tmp = head;
    for (int i = 0; i < index; i++) {
        if (tmp != null) {
            tmp = tmp.next;
        }
    }
    if (tmp != null) {
        tmp.data.tampilkanInformasi();
    } else {
        System.out.println(x: "Indeks tidak valid");
    }
}
```

```

3 public class SingleLinkedList20 {
104 // method indexOf
105 int indexOf(String key) {
106     Node20 temp = head;
107     int index = 0;
108     while (temp != null && !temp.data.nama.equalsIgnoreCase(key)) {
109         temp = temp.next;
110         index++;
111     }
112     if (temp != null) {
113         return -1; // Ditemukan, kembalikan indeks
114     } else {
115         return index; // Jika tidak ditemukan
116     }
117 }
118
119 // method removeFirst
120 void removeFirst() {
121     if (IsEmpty()) {
122         System.out.println(x: "Linked list kosong, tidak ada yang dihapus");
123     } else if (head == tail) { // Hanya ada satu node
124         head = null;
125         tail = null;
126     } else {
127         head = head.next; // Pindahkan head ke node berikutnya
128     }
129 }
130
131 // method removeLast
132 void removeLast() {
133     if (IsEmpty()) {
134         System.out.println(x: "Linked list kosong, tidak ada yang dihapus");
135     } else if (head == tail) { // Hanya ada satu node
136         head = null;
137         tail = null;
138     } else {
139         Node20 temp = head;
140         while (temp.next != tail) {
141             temp = temp.next; // Cari node sebelum tail
142         }
143         temp.next = null; // Putuskan hubungan dengan tail lama
144         tail = temp;      // Pindahkan tail ke node sebelumnya
145     }
146 }

```

```

public class SingleLinkedList20 {
    // method remove
    void remove(String key) {
        if (isEmpty()) {
            System.out.println(x: "Linked list kosong, tidak ada yang dihapus");
        } else {
            Node20 temp = head;
            while (temp.next != null) {
                if (temp.data.nama.equalsIgnoreCase(key) && (temp == head)) { // Jika node yang akan dihapus adalah head
                    this.removeFirst(); // Panggil method removeFirst untuk menghapus head
                    break;
                } else if (temp.next.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next; // Putuskan hubungan dengan node yang dihapus
                    if (temp.next == null) { // Jika node yang dihapus adalah tail
                        tail = temp; // Pindahkan tail ke node sebelumnya
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }

    // method removeAt
    void removeAt (int index) {
        if (index == 0) {
            removeFirst();
        } else {
            Node20 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next; // Putuskan hubungan dengan node yang dihapus
            if (temp.next == null) { // Jika node yang dihapus adalah tail
                tail = temp; // Pindahkan tail ke node sebelumnya
            }
        }
    }
}

```

2. Hasil Modifikasi program di SLLMain20.java

```

public class SLLMain20 {
    public static void main(String[] args) {

        // penghapusan & pengaksesan data
        System.out.println(x: "data index 1: ");
        sll.getData(index: 1);

        System.out.println("data mahasiswa an Bimon berada pada index: " + sll.indexOf(key: "Bimon"));
        System.out.println();

        sll.removeFirst(); // Menghapus Alvaro
        sll.removeLast(); // Menghapus Dirga
        sll.print(); // Cetak hasil akhir setelah penghapusan
        sll.removeAt(index: 0); // Menghapus Cintia (sekarang di indeks 0 setelah penghapusan sebelumnya)
        sll.print(); // Cetak hasil akhir setelah penghapusan Cintia
    }
}

```


Hasil output pada main

```
data index 1:
NIM : 21212203
Nama : Dirga
Kelas : 4D
IPK : 3.6
-----
data mahasiswa an Bimon berada pada index: 2

Isi Linked List:
NIM : 21212203
Nama : Dirga
Kelas : 4D
IPK : 3.6
-----
NIM : 23212201
Nama : Bimon
Kelas : 2B
IPK : 3.8
-----
Isi Linked List:
NIM : 23212201
Nama : Bimon
Kelas : 2B
IPK : 3.8
-----
```

Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!

Jawab: Keyword break pada fungsi remove() digunakan untuk menghentikan perulangan (while) setelah data yang dicari berhasil ditemukan dan dihapus. Hal ini dilakukan agar program tidak terus mencari data lain, lebih efisien, dan menghindari error pada linked list.

2. Jelaskan kegunaan kode dibawah pada method remove

```
temp.next = temp.next.next; //
if (temp.next == null) { // Jik
    tail = temp; // Pindahkan t
}
```

Jawab: Pada potongan kode ini:

```
temp.next = temp.next.next; //
```

Kegunaannya:

- Memutus node yang ingin dihapus dari linked list.
- Menghubungkan node sebelumnya langsung ke node setelahnya.

Pada potongan kode ini:

```
if (temp.next == null) { // Jika  
    tail = temp; // Pindahkan t
```

Kegunaannya:

- Mengecek apakah node yang dihapus adalah node terakhir (tail).
- Jika iya, maka tail dipindahkan ke node sebelumnya (temp) agar tail tetap benar menunjuk node terakhir linked list.

C. TUGAS

Buatlah implementasi program antrian layanan unit kemahasiswaan sesuai dengan berikut ini :

- Implementasi antrian menggunakan Queue berbasis Linked List!
- Program merupakan proyek baru bukan modifikasi dari percobaan
- Ketika seorang mahasiswa akan mengantri, maka dia harus mendaftarkan datanya
- Cek antrian kosong, Cek antrian penuh, Mengosongkan antrian.
- Menambahkan antrian
- Memanggil antrian
- Menampilkan antrian terdepan dan antrian paling akhir
- Menampilkan jumlah mahasiswa yang masih mengantre.

Jawab:

1. Hasil program di Mahasiswa20.java

Class Mahasiswa digunakan untuk menyimpan data mahasiswa yang akan mengantre.

```
Minggu11 > TugasJB11 > J Mahasiswa20.java > ...
1  package Minggu11.TugasJB11;
2
3  public class Mahasiswa20 {
4      String nim;
5      String nama;
6      String jurusan;
7
8      Mahasiswa20(String nim, String nama, String jurusan) {
9          this.nim = nim;
10         this.nama = nama;
11         this.jurusan = jurusan;
12     }
13
14     void tampilData() {
15         System.out.println("NIM      : " + nim);
16         System.out.println("Nama    : " + nama);
17         System.out.println("Jurusan : " + jurusan);
18     }
19 }
20
```

2. Hasil program di NodeMhs20.java

Class Node digunakan sebagai elemen pada Linked List.

- data → menyimpan objek mahasiswa
- next → menunjuk node berikutnya

```
Minggu11 > TugasJB11 > J NodeMhs20.java > Language Support for Java(TM) by Red Hat > No
1  package Minggu11.TugasJB11;
2
3  public class NodeMhs20 {
4      Mahasiswa20 data;
5      NodeMhs20 next;
6
7      NodeMhs20(Mahasiswa20 data) {
8          this.data = data;
9          this.next = null;
10     }
11 }
```

3. Hasil program di QueueSingleLinkedList.java

Class ini merupakan implementasi queue menggunakan linked list.

- front → menunjuk antrian paling depan
- rear → menunjuk antrian paling belakang
- size → jumlah data dalam antrian
- max → kapasitas maksimal antrian

```
Minggu11 > TugasJ811 > J QueueLinkedList.java > Java > QueueLinkedList
1  package Minggu11.TugasJ811;
2
3  public class QueueLinkedList {
4      NodeMhs20 front;
5      NodeMhs20 rear;
6      int size;
7      int max;
8
9      QueueLinkedList(int max) {
10         this.max = max;
11         front = rear = null;
12         size = 0;
13     }
14
```

Method isEmpty()

Digunakan untuk mengecek apakah antrian kosong. Jika front == null, berarti tidak ada data dalam queue.

```
15         // cek kosong
16         boolean isEmpty() {
17             return front == null;
18         }
19
```

Method isFull()

Digunakan untuk mengecek apakah antrian penuh. Jika jumlah data sama dengan kapasitas maksimum, maka antrian penuh.

```
20         // cek penuh
21         boolean isFull() {
22             return size == max;
23         }
24
```

Method enqueue()

- Membuat node baru
- Jika antrian kosong → front dan rear menunjuk node baru
- Jika tidak kosong → node baru ditambahkan di belakang
- rear dipindah ke node terakhir
- size bertambah 1

```
25 // tambah antrian
26 void enqueue(Mahasiswa20 mhs) {
27     if (isFull()) {
28         System.out.println(x: "Antrian penuh!");
29         return;
30     }
31
32     NodeMhs20 baru = new NodeMhs20(mhs);
33
34     if (isEmpty()) {
35         front = rear = baru;
36     } else {
37         rear.next = baru;
38         rear = baru;
39     }
40
41     size++;
42     System.out.println(x: "Mahasiswa berhasil masuk antrian.");
43 }
```

Method dequeue()

Digunakan untuk memanggil atau menghapus antrian terdepan.

- Menampilkan data mahasiswa paling depan
- front dipindah ke node berikutnya
- Size dikurangi 1
- Jika antrian kosong → rear = null

```

3   public class QueueLinkedList {
45      // panggil antrian
46      void dequeue() {
47          if (isEmpty()) {
48              System.out.println(x: "Antrian kosong!");
49              return;
50          }
51
52          System.out.println(x: "Mahasiswa dipanggil:");
53          front.data.tampilData();
54
55          front = front.next;
56          size--;
57
58          if (front == null) {
59              rear = null;
60          }
61      }
62

```

Method peekFront()

Digunakan untuk melihat data mahasiswa paling depan tanpa menghapus data.

```

63      // tampil depan
64      void peekFront() {
65          if (isEmpty()) {
66              System.out.println(x: "Antrian kosong!");
67          } else {
68              System.out.println(x: "Antrian Terdepan:");
69              front.data.tampilData();
70          }
71      }
72

```

Method peekRear()

Digunakan untuk melihat data mahasiswa paling belakang.

```

72      // tampil belakang
73      void peekRear() {
74          if (isEmpty()) {
75              System.out.println(x: "Antrian kosong!");
76          } else {
77              System.out.println(x: "Antrian Paling Akhir:");
78              rear.data.tampilData();
79          }
80      }
81

```

Method printQueue()

Digunakan untuk menampilkan seluruh isi antrian dari depan ke belakang.

```
3 public class QueueLinkedList {
82
83     // tampil semua antrian
84     void printQueue() {
85         if (isEmpty()) {
86             System.out.println(x: "Antrian kosong!");
87             return;
88         }
89
90         NodeMhs20 temp = front;
91
92         System.out.println(x: "=== DAFTAR ANTRIAN ===");
93
94         while (temp != null) {
95             temp.data.tampilData();
96             System.out.println(x: "-----");
97             temp = temp.next;
98         }
99     }
}
```

Method jumlahAntrian()

Digunakan untuk menampilkan jumlah mahasiswa yang masih mengantre.

```
100
101     // jumlah antrian
102     void jumlahAntrian() {
103         System.out.println("Jumlah mahasiswa dalam antrian: " + size);
104     }
105 }
```

Method clear()

Digunakan untuk menghapus seluruh isi antrian.

```
106     // kosongkan antrian
107     void clear() {
108         front = rear = null;
109         size = 0;
110
111         System.out.println(x: "Antrian berhasil dikosongkan.");
112     }
113 }
```

4. Hasil program di MainQueueSLL20.java

```
package Minggu11.TugasJB11;
import java.util.Scanner;

public class MainQueueSLL20 {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        QueueLinkedList antrian = new QueueLinkedList(max: 10);

        int pilih;

        do {
            System.out.println(x: "\n=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN =");
            System.out.println(x: "1. Tambah Antrian");
            System.out.println(x: "2. Panggil Antrian");
            System.out.println(x: "3. Tampilkan Semua Antrian");
            System.out.println(x: "4. Tampilkan Antrian Terdepan");
            System.out.println(x: "5. Tampilkan Antrian Paling Akhir");
            System.out.println(x: "6. Jumlah Mahasiswa Mengantre");
            System.out.println(x: "7. Cek Antrian Kosong");
            System.out.println(x: "8. Cek Antrian Penuh");
            System.out.println(x: "9. Kosongkan Antrian");
            System.out.println(x: "0. Keluar");
            System.out.print(s: "Pilih menu: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    if (antrian.isFull()) {
                        System.out.println(x: "Antrian sudah penuh!");
                    } else {
                        System.out.print(s: "Masukkan NIM      : ");
                        String nim = sc.nextLine();

                        System.out.print(s: "Masukkan Nama      : ");
                        String nama = sc.nextLine();

                        System.out.print(s: "Masukkan Jurusan   : ");
                        String jurusan = sc.nextLine();

                        Mahasiswa20 mhs = new Mahasiswa20(nim, nama, jurusan);

                        antrian.enqueue(mhs);
                    }
                    break;
            }
        } while (true);
    }
}
```



```

4   public class MainQueueSLL20 {
6       public static void main(String[] args) {

50         case 2:
51             antrian.dequeue();
52             break;
53
54         case 3:
55             antrian.printQueue();
56             break;
57
58         case 4:
59             antrian.peekFront();
60             break;
61
62         case 5:
63             antrian.peekRear();
64             break;
65
66         case 6:
67             antrian.jumlahAntrian();
68             break;
69
70         case 7:
71             if (antrian.isEmpty()) {
72                 System.out.println(x: "Antrian kosong.");
73             } else {
74                 System.out.println(x: "Antrian tidak kosong.");
75             }
76             break;
77
78         case 8:
79             if (antrian.isFull()) {
80                 System.out.println(x: "Antrian penuh.");
81             } else {
82                 System.out.println(x: "Antrian belum penuh.");
83             }
84             break;
85
86         case 9:
87             antrian.clear();
88             break;
89
90         case 0:
91             System.out.println(x: "Program selesai.");
92             break;
93
94         default:
95             System.out.println(x: "Menu tidak tersedia!");
96     }
97
98     } while (pilih != 0);
99
100    sc.close();

```

Hasil Output di Main

```
=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar
Pilih menu: 1
Masukkan NIM      : 123
Masukkan Nama     : Budi
Masukkan Jurusan  : TI
Mahasiswa berhasil masuk antrian.
```

```
=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar
Pilih menu: 1
Masukkan NIM      : 124
Masukkan Nama     : Rudi
Masukkan Jurusan  : AN
Mahasiswa berhasil masuk antrian.
```

```
=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar
Pilih menu: 1
Masukkan NIM      : 125
Masukkan Nama     : TIA
Masukkan Jurusan  : AK
Mahasiswa berhasil masuk antrian.
```

```
=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar
Pilih menu: 3
=== DAFTAR ANTRIAN ===
NIM      : 123
Nama     : Budi
Jurusan  : TI
-----
NIM      : 124
Nama     : Rudi
Jurusan  : AN
-----
NIM      : 125
Nama     : TIA
Jurusan  : AK
-----
```

```
=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===
1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar
Pilih menu: 2
Mahasiswa dipanggil:
NIM      : 123
Nama     : Budi
Jurusan  : TI
```

=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar

Pilih menu: 4

Antrian Terdepan:

NIM : 124

Nama : Rudi

Jurusan : AN

=== SISTEM ANTRIAN LAYANAN KEMAHASISWAAN ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Tampilkan Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Jumlah Mahasiswa Mengantre
7. Cek Antrian Kosong
8. Cek Antrian Penuh
9. Kosongkan Antrian
0. Keluar

Pilih menu: 5

Antrian Paling Akhir:

NIM : 125

Nama : TIA

Jurusan : AK